

FLEETGUARD AI

Code Documentation — Version 3

*A complete, from-scratch guide to the Python, Machine Learning, and Aerospace concepts behind this project — written for a reader with **no prior background in any of the three**.*

Companion to: `src_v2/` (the FleetGuard AI v2 codebase), including the realistic-telemetry update described in Chapter 6.

Also see: `FleetGuard_AI_Architecture.pdf` (visual pipeline diagram + demo runbook) and `README_v2.md` (project overview) in this same `outputs_v2/` folder.

If a word is unfamiliar anywhere in this document, check the Glossary (Chapter 8) — every technical term used is defined there.

Contents

1. Introduction & How to Use This Document
2. Aerospace Concepts You Need
3. Machine Learning Concepts You Need
4. Python Concepts You Need to Read This Code
5. Project Architecture, in Plain English
6. File-by-File Code Walkthrough
7. How to Run the Demo
8. Glossary (every term, alphabetical)
9. Frequently Asked Questions

1. Introduction & How to Use This Document

This document explains the FleetGuard AI aerospace predictive-maintenance project from absolute zero. It assumes you have never written Python, never trained a machine learning model, and have never worked on aircraft maintenance. If you already know one or two of those three areas, skip ahead — each chapter stands mostly on its own.

How this document is organized

- **Chapters 2–4** teach the background concepts you need — aerospace, then machine learning, then Python — before you look at a single line of the project's code.
- **Chapter 5** gives a one-page mental model of how the whole system fits together.
- **Chapter 6** is the core of this document: every file in `src_v2/`, explained function by function, in plain English, with the real code shown alongside the explanation.
- **Chapter 7** is a condensed run-it-yourself guide.
- **Chapter 8** is an alphabetical glossary of every technical term used anywhere in this document — jump there any time a word doesn't make sense.
- **Chapter 9** answers questions a beginner typically asks after a first read-through.

Reading tip: the first time a technical term is used in this document, it appears in **bold**. Every bolded term is defined in the Glossary.

2. Aerospace Concepts You Need

2.1 What is predictive maintenance?

There are three broad strategies for maintaining a machine (a car, a factory robot, an aircraft part):

- **Reactive maintenance** — you fix it after it breaks. Cheap until it breaks at the worst possible time.
- **Scheduled (preventive) maintenance** — you replace or inspect the part on a fixed calendar or usage interval (e.g. every 5,000 miles), whether or not it actually needs it yet. Safe, but wastes good remaining life on parts that were still healthy.
- **Predictive maintenance** — you continuously measure the part's actual condition (via sensors) and fix it exactly when the data says it's starting to wear out, not before and not after. This project is a predictive maintenance system.

Analogy: a car's fixed oil-change schedule is **scheduled** maintenance. A dashboard light that says “oil quality degrading, service within 200 miles” based on an actual sensor reading is **predictive** maintenance.

2.2 Vocabulary: fleet, aircraft, component, unit, cycle

- **Fleet** — every aircraft this system watches (40 aircraft in this project's simulation).
- **aircraft_id** — one physical airplane, e.g. AC-014.
- **Component** — a category of part being tracked: engine, landing_gear, brakes, or bogie.
- **unit_id** — one specific, physical part on one specific aircraft, e.g. AC-014-landing_gear-54. Every aircraft has its own landing gear, engine, etc. — each with its own independent wear history.
- **Cycle** — one flight (one takeoff plus one landing). Wear is measured in cycles, not calendar time, because mechanical wear tracks *usage*, not the passage of days.

2.3 The four components this system watches

- **Engine** — produces thrust. Its health is classically monitored through Exhaust Gas Temperature (rises as internal parts wear), vibration (bearing wear shows up here earliest), and oil debris (a magnetic “chip detector” catches metal shavings shed by a wearing bearing before it fails outright).
- **Landing gear** — the strut assembly that absorbs landing shock. Its oleo strut is pressurized with hydraulic fluid/nitrogen; a slow leak shows up as declining pressure, and worn shock absorption shows up as rising vibration.
- **Brakes** — slow the aircraft on landing/taxi. Brake temperature and hydraulic system pressure are the primary health signals; brakes that run hotter than normal for a given landing are a classic early-wear sign.

- **Bogie** — the wheel/axle assembly landing gear sits on. Wheel bearing wear shows up as vibration and rising bearing temperature.

2.4 What the five sensor channels actually measure

Real transport-category aircraft record roughly **88 parameters** throughout every flight (mandated by FAA Advisory Circular 20-141B / the ARINC 717 data standard) — things like altitude, engine temperatures, vibration, and pressures. This project's simulator uses 5 channels as a simplified stand-in for that same family of real measurements:

vibration_mm_s	Vibration velocity in millimeters/second. The earliest and most sensitive warning sign of bearing wear — real aerospace vibration monitoring (and the general ISO 10816 industrial standard) relies on exactly this kind of measurement.
temperature_c	Degrees Celsius. Exhaust Gas Temperature for the engine; brake or bearing temperature for the other three components.
pressure_psi	Pounds per square inch. Oil pressure (engine), strut/hydraulic pressure (landing gear, brakes), or tire pressure (bogie).
oil_debris_ppm	Parts per million of metal debris detected in lubricating oil by a chip detector — a real, physical early-warning sensor for internal wear.
rotational_speed_deviation_pct	How far a part's rotational speed deviates (in percent) from what's expected for its current operating setting — generalizes the real N1/N2 engine-speed monitoring concept to any rotating part.

Important honesty note: these baseline values and thresholds are engineering-plausible reference points grounded in publicly documented standards (ISO 10816 vibration zones, ~3000 psi as a standard commercial-aircraft hydraulic pressure, typical turbofan EGT ranges), **not** exact manufacturer specifications for any one aircraft type, and are **not** derived from any accident investigation data. Real flight-recorder data from crash investigations isn't published as raw datasets, and this project does not attempt to reproduce any real incident's telemetry.

2.5 RUL — Remaining Useful Life

RUL answers one question: “how many more flight cycles can this exact physical part safely operate before it should be replaced?” It's a moving target that counts down by one every cycle. A maintenance team that knows a part's RUL can schedule its replacement for the next convenient ground time — instead of being surprised by a failure mid-fleet-operation.

2.6 MRO and work orders

MRO stands for Maintenance, Repair, and Overhaul — the team or system that physically inspects, fixes, or replaces aircraft parts. A **work order** is a formal, trackable instruction telling MRO exactly what needs attention: which part, on which aircraft, how urgently, and why.

3. Machine Learning Concepts You Need

3.1 What is a “dataset” here?

A dataset is just a table: rows and columns. In this project, **one row** = one sensor reading, for one physical part, at one flight cycle. **Columns** split into two kinds: **features** (the inputs the model is allowed to look at — sensor readings) and **labels** (the correct answer we want the model to learn to predict, e.g. the true RUL).

3.2 Supervised vs. unsupervised learning

- **Supervised learning** (used for RUL prediction): during training, we already know the correct answer for every example (the simulator knows each part's true total lifespan, so the true RUL at every cycle is computable). The model learns the mapping from features → correct answer.
- **Unsupervised learning** (used for anomaly detection): we do *not* tell the model which readings are “bad”. It only learns what normal readings typically look like, then flags anything that doesn't resemble that normal pattern.

3.3 Regression vs. anomaly detection

- **Regression** — predict a number. This project uses regression to predict RUL (a number of cycles).
- **Anomaly detection** — decide whether something looks normal or unusual, without needing hand-labeled examples of every possible way a part can fail.

3.4 Decision Trees and Random Forests

A **Decision Tree** is a flowchart of yes/no questions about the data that ends in a prediction — for example: “if vibration_mean is above X and temperature is rising, predict RUL near 20 cycles.” A single tree, trained on one dataset, tends to overreact to quirks in that specific data.

A **Random Forest** trains hundreds of slightly different decision trees — each one sees a random subset of the training rows and a random subset of the features — and then **averages** all of their individual predictions together. Averaging many independently-wrong-in-different-ways trees cancels out each tree's individual mistakes, producing a far more reliable prediction than any single tree could give.

Analogy: asking 300 independent mechanics for their best guess and averaging the answers, instead of trusting the opinion of just one mechanic.

3.5 Isolation Forest

Isolation Forest is the algorithm this project uses for anomaly detection. Its core idea: an unusual data point is *easier to isolate* than a normal one. Imagine repeatedly splitting your data in half at random thresholds — a point that already sits apart from the crowd gets separated from everything else in very few random splits, while a typical, “in the middle of the pack” point takes many more splits to isolate. Isolation Forest builds many such random-split trees and measures, on average, how few splits it took to isolate each point. Few splits needed → likely an anomaly.

3.6 Feature engineering: rolling-window statistics

A single instantaneous sensor reading is noisy and can be misleading on its own. This project computes, for every sensor, the trailing **mean**, **standard deviation** (spread), **min**, **max**, and **slope** (rate of change) over the last 5 cycles. The slope is especially valuable: a rising trend is an early warning sign of developing wear, visible before the raw sensor value itself looks clearly abnormal.

3.7 Train / Validation / Test split — and the leakage trap

Before training, the dataset is divided three ways: **training** data (the model learns from this), **validation** data (used to tune settings, like the anomaly-alert threshold, on data the model wasn't fitted on), and **test** data (a final, untouched check of real-world accuracy).

The critical, easy-to-get-wrong detail: this project splits by **unit_id** (a whole physical part's entire history goes into exactly one split), never by individual row. If cycle 41 of a part landed in training and cycle 42 of the very same part landed in test, the model would effectively be tested on data almost identical to what it already memorized — producing misleadingly good accuracy that won't hold up on a genuinely new part. This mistake is called **data leakage**.

3.8 Overfitting, briefly

A model has **overfit** when it has effectively memorized quirks of its training data instead of learning patterns that generalize. An overfit model looks great on training data and disappoints on new data. Holding out validation/test splits (3.7) is exactly how you catch this before it becomes a costly surprise in production.

3.9 How do we know if the model is any good? (Metrics)

MAE	Mean Absolute Error — on average, how many cycles off is a RUL prediction? Lower is better.
RMSE	Root Mean Squared Error — similar to MAE, but penalizes large individual misses more heavily. Lower is better.
Recall	Of all the real anomalies that existed, what fraction did the model actually catch? Higher is better.
Precision	Of everything the model flagged as anomalous, what fraction was a genuine anomaly? Higher is better.

False alarm rate

How often the model flags a perfectly normal reading as anomalous (“crying wolf”). Lower is better.

4. Python Concepts You Need to Read This Code

4.1 Variables, functions, and imports

A **variable** gives a name to a value (`total_life = 250`). A **function** is a named, reusable block of code that takes inputs and returns an output. An **import** statement pulls in code written in another file so it can be reused.

```
def evaluate(y_true, y_pred):
    mae = float(np.mean(np.abs(y_true - y_pred)))
    return {"MAE": round(mae, 2)}

# calling the function:
score = evaluate(actual_values, predicted_values)
```

This defines a function named `evaluate` that takes two inputs and returns a **dictionary** (see below) containing the result.

4.2 Dictionaries and lists

- A **list** is an ordered collection: `["engine", "brakes"]`.
- A **dictionary** maps keys to values, like a lookup table: `{"engine": 620, "brakes": 140}` — this project uses dictionaries constantly, e.g. one fitted model *per component*, keyed by component name.

4.3 pandas DataFrames in 60 seconds

pandas is Python's standard library for working with tables. A **DataFrame** is a table with named columns. The most common operations you'll see in this codebase:

```
df["vibration_mm_s"]          # select one column
df[df["component"] == "engine"] # select rows where a condition is true
df.groupby("unit_id")         # group rows by unit, to compute per-part stats
df.sort_values("cycle")       # sort rows by a column
```

4.4 numpy arrays in 60 seconds

numpy is Python's library for fast numerical arrays and math. `np.mean(x)`, `np.clip(x, 0, None)` (force values to stay within a range), and `rng.normal(...)` (draw random noise from a bell-curve distribution) appear throughout this project.

4.5 This project's specific coding style

- No custom classes — just plain functions and dictionaries. A collection of “one fitted model per component” is simply `{"engine": <model>, "brakes": <model>, ...}`.
- Comments explain **why** a choice was made, not just what the code does — read them, they carry real design reasoning.
- Every file has a short header comment at the top explaining its overall purpose before any code starts.

5. Project Architecture, in Plain English

The system is split into two halves, matching a standard real-world production pattern: training is slow and occasional; serving is fast and frequent.

5.1 train.py — the offline, occasional half

Run this whenever there's meaningfully new historical data. It: (1) generates/loads fleet telemetry, (2) engineers rolling-window features, (3) trains one Isolation Forest + one Random Forest per component, (4) tunes each component's alert threshold on the validation split and evaluates on the test split, (5) saves everything to disk.

5.2 predict_service.py — the fast, frequent half

Run this every time new telemetry needs scoring (in a real deployment: nightly, or after every flight). It **never retrains** — it just loads the models train.py already saved, scores the new batch, builds the dashboard, and creates maintenance work orders.

Why split them apart? training 4 Random Forests + 4 Isolation Forests takes roughly 10–15 seconds; a real fleet might need scoring every few minutes. Retraining that often would be wasteful and pointlessly slow — so serving is built to depend only on *already-trained* models, never on retraining.

See the companion FleetGuard_AI_Architecture.pdf in this folder for a visual diagram of this same flow.

6. File-by-File Code Walkthrough

This chapter covers every file in `src_v2/`, in the order data flows through them. Each section shows the file's real code alongside a plain-English explanation.

6.1 `config.py` — every tunable setting, in one place

Keeping every constant (fleet size, model hyperparameters, file paths) in a single file is standard production practice: changing a setting never requires hunting through model code.

```
N_AIRCRAFT = 40
COMPONENTS = ["engine", "landing_gear", "brakes", "bogie"]
SENSOR_COLUMNS = ["vibration_mm_s", "temperature_c", "pressure_psi",
                  "oil_debris_ppm", "rotational_speed_deviation_pct"]
IFOREST_N_ESTIMATORS = 200      # how many trees in the anomaly-detection forest
RF_N_ESTIMATORS = 300          # how many trees in the RUL-prediction forest
```

Beginner note: changing `N_AIRCRAFT` to 100 and re-running `train.py` is a safe, easy way to see how the rest of the pipeline responds to more data — nothing else needs to change.

6.2 `data_simulator_v2.py` — generates the fleet telemetry

Since this project has no access to a real airline's sensor feed, it generates realistic synthetic data instead. For every simulated part, it picks a random total lifespan, then grows each of the 5 sensor readings away from a healthy baseline as the part ages — using a **different curve shape per sensor** so that vibration and oil debris rise earlier than temperature does, matching how real bearing wear is actually detected (Section 2.4).

```
WEAR_EXPONENT = {
    "vibration_mm_s": 1.3, "oil_debris_ppm": 1.4,
    "rotational_speed_deviation_pct": 1.8,
    "pressure_psi": 2.0, "temperature_c": 2.5,
}

for sensor in SENSOR_COLUMNS:
    wear = (cycles / total_life) ** WEAR_EXPONENT[sensor]
    trend = base[sensor] * DEGRADE_MAGNITUDE_FRACTION[sensor] * wear
    data[sensor] = base[sensor] + trend + rng.normal(0, noise[sensor], size=len(cycles))
```

A lower exponent (vibration = 1.3) makes wear climb away from zero earlier in a part's life; a higher exponent (temperature = 2.5) keeps it near zero longer, then rises sharply only near end-of-life — exactly the “vibration warns first, temperature warns last” pattern documented in real aerospace maintenance literature.

Every row also gets a realistic `timestamp_utc`: each part's most recent cycle lands close to the current date, and earlier cycles step backward in time at a simulated 2–5 flights/day pace — so the CSV reads like a live fleet feed rather than an abstract counter.

Key functions: `make_one_unit_data()` builds one part's full history; `simulate_fleet()` calls it once per aircraft x component combination; `split_units()` randomly assigns whole units (never individual rows) to train/val/test (Section 3.7).

6.3 features.py — turns raw readings into model-ready features

```
for sensor in SENSOR_COLUMNS:
    roll = grouped[sensor].rolling(window, min_periods=1)
    df[f"{sensor}_mean"] = roll.mean()...
    df[f"{sensor}_std"] = roll.std()...
    df[f"{sensor}_slope"] = ... # trailing rate-of-change, via straight-line fit
```

For every sensor, this adds 5 new columns computed over the trailing 5 cycles: mean, standard deviation, min, max, and slope (Section 3.6). `feature_column_names()` returns the exact, fixed-order list of all resulting columns — both training and serving code must use this same order, or a model would be fed the wrong numbers into the wrong slots.

6.4 anomaly_detection_v2.py — the Isolation Forest half

```
for component, comp_df in featured_train_df.groupby("component"):
    X = comp_df[cols].values
    model = IsolationForest(n_estimators=200, contamination=0.05, ...)
    model.fit(X)
    models[component] = model
```

`train_anomaly_models()` fits one Isolation Forest per component, using only training-split rows. `compute_thresholds()` then picks each component's alert cutoff using the **validation** split — not sklearn's own built-in cutoff — which is the more rigorous, production-realistic choice (Section 3.7). `score_anomalies()` applies the right component's model + threshold to every row.

Beginner note: scikit-learn's `score_samples()` returns *lower* values for more abnormal points — the opposite of what you'd intuitively expect. This file negates the sign once, in one place, so everywhere else in the project, higher score always means more anomalous.

6.5 rul_model_v2.py — the Random Forest half

```
model = RandomForestRegressor(n_estimators=300, max_depth=12, ...)
model.fit(X, y) # y = the true RUL column
...
predictions = np.clip(predictions, 0, None) # RUL can never be negative
```

Structurally almost identical to the anomaly-detection file, just with a regression model instead of an anomaly-detection model, and real target labels (`y = true RUL`) to learn from.

`get_feature_importances()` exposes, for free, which engineered features each Random Forest actually relied on most — useful for sanity-checking that the model learned something physically sensible.

6.6 evaluation_v2.py — scores the models on held-out data

Computes the metrics defined in Section 3.9 (MAE, RMSE, recall, precision, false alarm rate), plus one maintenance-specific metric: **detection lead time** — for every part that was eventually flagged, how many cycles before its actual end-of-life did the first alert fire? A large lead time means the system gives maintenance planners real runway, not a last-minute warning.

6.7 model_registry.py — saves and loads trained models

```
joblib.dump(model, f"{artifact_dir}/{name}_{component}.joblib")
...
card[name] = {"trained_at_utc": ..., "hyperparameters": ..., "metrics": ...}
json.dump(card, open(f"{artifact_dir}/model_card.json", "w"))
```

joblib is scikit-learn's recommended tool for saving a fitted model to disk (more efficient than Python's generic pickle for objects holding large numeric arrays). Alongside the model files, this writes a JSON “model card” recording exactly what was trained, when, with what settings, and how well it scored — a full audit trail from raw data to deployed model.

6.8 dashboard_v2.py — fleet health, prioritized

```
def classify_health(row):
    if row["predicted_RUL"] <= CRITICAL_RUL:      return "Critical"
    elif row["predicted_RUL"] <= WARNING_RUL or row["is_anomaly"] == 1:
        return "Warning"
    else:                                          return "Healthy"
```

Takes only each part's *latest* cycle, labels it Healthy/Warning/Critical, ranks everything by urgency, and writes an HTML dashboard file (a plain, self-contained webpage — open it directly in any browser, no server required).

6.9 mro_integration_v2.py — auto-generated work orders

```
"work_order_id": f"W0-{row['unit_id']}-{row['cycle']}", # doubles as an idempotency key
```

Every part flagged Warning or Critical becomes a work order. Two production-hardening details worth knowing: (1) each work order's ID is also an **idempotency key** — if the same work order gets sent twice (e.g. after a retry), a real MRO system recognizes the duplicate ID and doesn't create two tickets for the same problem; (2) sending is wrapped in retry logic, simulating how a real network call to an external system occasionally fails and must not simply give up after one attempt.

6.10 train.py — the offline training entry point

Runs the entire training-side pipeline end to end: generate data → engineer features → train both model types → tune thresholds → evaluate on the test split → save everything. This is the file you run whenever you want a fresh, retrained model.

6.11 predict_service.py — the serving entry point

Loads the models train.py already saved (never retrains), scores a fresh telemetry batch, builds the dashboard, and sends work orders. This is the file that would run on a schedule in a real deployment.

6.12 tests/test_models.py — automated correctness checks

Five automated tests, run with Python's built-in `unittest` module (no extra install needed). They check things like: does a rising sensor trend produce a positive slope; does the anomaly pipeline only ever output 0 or 1; does the trained RUL model actually beat a naive “always guess the average” baseline. Running these after any code change is a fast way to catch a broken assumption before it reaches real use.

7. How to Run the Demo

From inside the project's `src_v2/` folder, in order:

```
pip install -r requirements_v2.txt
python train.py
python predict_service.py
python -m unittest tests.test_models -v
```

- **pip install** — installs numpy, pandas, scikit-learn, and joblib. No GPU, no API keys, no internet needed after this step.
- **train.py** (~11 seconds) — generates data, trains all models, saves them to `artifacts/`.
- **predict_service.py** (~7 seconds) — loads the saved models, scores a new batch, writes the dashboard and work orders to `outputs_v2/`.
- **unittest** (~2 seconds) — confirms all 5 automated tests still pass.

See `FleetGuard_AI_Architecture.pdf` (page 2) in this same folder for the full visual runbook, including exactly where each output file lands.

8. Glossary

Aircraft_id	Identifier for one physical airplane, e.g. AC-014.
Anomaly detection	Deciding whether a reading looks normal or unusual, without needing labeled examples of every possible failure.
Anomaly score	A number output by the Isolation Forest; higher means more unusual.
Bogie	The wheel/axle assembly that landing gear sits on.
Component	A category of part tracked by this system: engine, landing_gear, brakes, or bogie.
Cycle	One flight (one takeoff plus one landing); the unit wear is measured in.
Data leakage	When information from outside the training data (often a part's own future) accidentally influences training, producing misleadingly good accuracy.
DataFrame	A pandas table object: rows and named columns.
Decision tree	A flowchart of yes/no questions about data, ending in a prediction.
Dictionary (Python)	A collection that maps keys to values, e.g. {"engine": 620}.
EGT	Exhaust Gas Temperature — a key engine health parameter.
Feature	An input column the model is allowed to look at, e.g. vibration_mm_s.
Feature engineering	Computing new, more informative columns (like rolling averages) from raw data.
Fleet	Every aircraft/part this system tracks.
Idempotency key	A stable identifier that lets a system safely ignore an accidental duplicate request (e.g. a retried work order).
Isolation Forest	The anomaly-detection algorithm used here; flags points that are easy to isolate via random splits.
Joblib	A Python library for saving/loading fitted scikit-learn models to disk.
Label	The correct answer a supervised model is trained to predict, e.g. true RUL.
MAE	Mean Absolute Error — average size of a prediction's mistake, in cycles.
Model card	A JSON record of what was trained, when, with what settings, and how well it scored.

MRO	Maintenance, Repair, and Overhaul — the team/system that physically services aircraft parts.
Overfitting	When a model has memorized training data quirks instead of general patterns.
Precision	Of everything flagged as anomalous, the fraction that was a genuine anomaly.
Predictive maintenance	Fixing a part based on its actual measured condition, rather than a fixed schedule or waiting for failure.
Random Forest	Many decision trees trained on random subsets of data, averaged together.
Recall	Of all the real anomalies that existed, the fraction the model actually caught.
Regression	Predicting a number (e.g. RUL) rather than a category.
RMSE	Root Mean Squared Error — like MAE, but penalizes large misses more.
Rolling window	A trailing slice of the most recent N readings, used to compute stats like mean/slope.
RUL	Remaining Useful Life — how many more cycles a part can safely operate.
Scikit-learn	The Python machine-learning library providing IsolationForest and RandomForestRegressor.
Supervised learning	Training with known correct answers for every example.
Test split	Held-out data used only for a final, untouched accuracy check.
Train split	The data a model is actually fit on.
Unit_id	Identifier for one specific physical part on one specific aircraft.
Unsupervised learning	Training without labeled correct answers; the model learns what “normal” looks like on its own.
Validation split	Held-out data used to tune settings (like an alert threshold) without touching the test split.
Work order	A formal instruction telling MRO what part needs attention, and how urgently.

9. Frequently Asked Questions

Why simulated data instead of real airline sensor data?

Real fleet telemetry is proprietary to airlines and manufacturers, and real flight-recorder data from accident investigations isn't published as a raw, usable dataset — nor would it be appropriate to use a real tragedy as a demo dataset. This project's simulator generates unlimited, realistic, labeled data instead, letting the exact same modeling pipeline be pointed at real sensor data later without changing any model code.

Why does the anomaly detector have low precision (lots of false alarms)?

Its threshold is deliberately tuned to favor **recall** (catching real problems) over precision, on the reasoning that missing a genuine fault is far more costly than an extra false alarm in a safety-critical context. Calibrating precision further against real failure labels (rather than synthetic ones) is a natural next step, not a bug.

Can I add a 6th sensor channel?

Yes — add it to `SENSOR_COLUMNS` in `config.py` and give it a baseline/noise/wear entry in `data_simulator_v2.py`. `features.py`, the models, and the dashboard all read `SENSOR_COLUMNS` generically, so nothing else needs to change.

Where do I go next?

- `FleetGuard_AI_Architecture.pdf` — the visual pipeline diagram and demo runbook.
- `README_v2.md` — a shorter, high-level project overview.
- Re-read any confusing section of Chapters 2–4 after seeing the real code in Chapter 6 — concepts usually click faster the second time, once you've seen them in context.